
COOKIEGUARD

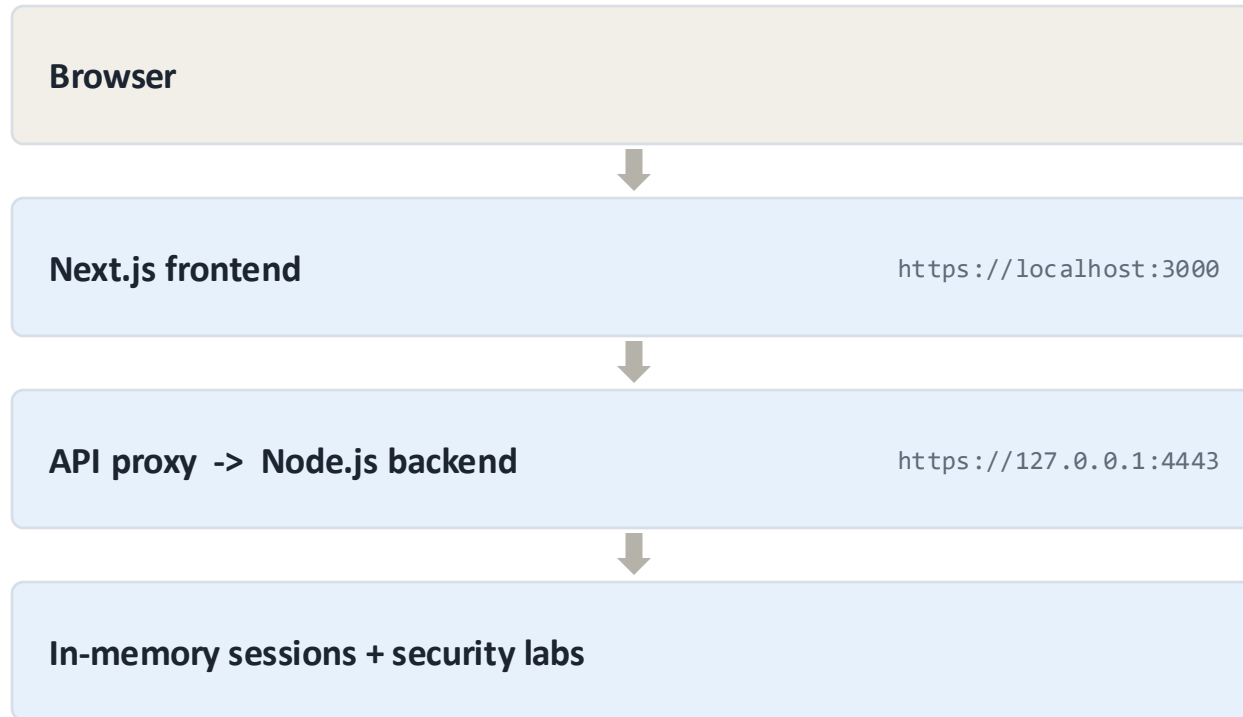
Interactive Cookie & Session Security Lab

Marco Ponce

M.S. Cybersecurity, Seton Hall University

App Security & Web Technology

Architecture & authentication



- Fixed demonstration credentials (demo / cookieguard) for repeatable testing
- Sessions are stored in memory, not persisted to a database
- Frontend forwards `/api/*` requests to the backend over HTTPS via a proxy route
- Local trusted certificates are generated with mkcert

Cookie security

cookieguard_session

Secure	Enabled
HttpOnly	Enabled
SameSite	Lax
Path	/
Expiration	Session
Persistent	Disabled
Domain	Host-only (not set)

Secure: Restricts the cookie to HTTPS transport.

HttpOnly: Limits direct JavaScript access to the cookie value.

SameSite: Controls whether the cookie is sent with cross-site requests.

XSS and HttpOnly

Vulnerable mode

EXPERIMENT 01 · COOKIE ACCESS

HttpOnly + XSS

Use the same controlled payload twice. The only meaningful change is whether the demonstration cookie is readable by JavaScript.

This lab uses a separate demonstration cookie. The authenticated CookieGuard session is never intentionally weakened.

01

Choose the control

Set the lab cookie to vulnerable or protected mode.

Vulnerable

Protected

Current mode: vulnerable

02

Run the payload

The payload attempts to read `document.cookie`.

Run XSS payload

JavaScript-visible cookies

cookieguard_xss_lab=lab-65c486bfc096cf0a57c2d5df4fa4f242

COMPARE

What should change?

Vulnerable	Protected
The lab cookie lacks <code>HttpOnly</code> , so the controlled payload can read it through JavaScript.	The lab cookie has <code>HttpOnly</code> , so the same payload cannot read that cookie.

Clear lab cookie

Protected mode

EXPERIMENT 01 · COOKIE ACCESS

HttpOnly + XSS

Use the same controlled payload twice. The only meaningful change is whether the demonstration cookie is readable by JavaScript.

This lab uses a separate demonstration cookie. The authenticated CookieGuard session is never intentionally weakened.

01

Choose the control

Set the lab cookie to vulnerable or protected mode.

Vulnerable

Protected

Current mode: protected

02

Run the payload

The payload attempts to read `document.cookie`.

Run XSS payload

JavaScript-visible cookies

COMPARE

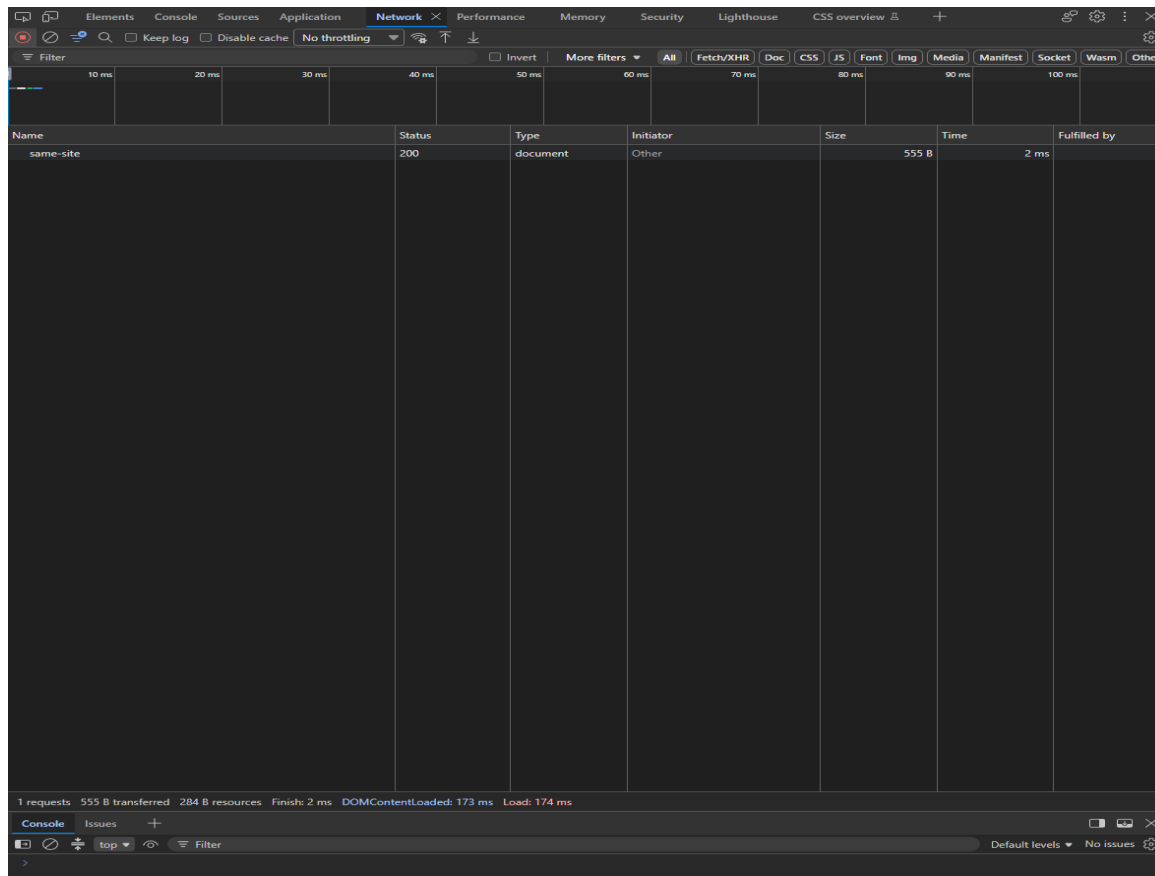
What should change?

Vulnerable	Protected
The lab cookie lacks <code>HttpOnly</code> , so the controlled payload can read it through JavaScript.	The lab cookie has <code>HttpOnly</code> , so the same payload cannot read that cookie.

Clear lab cookie

CSRF and SameSite

Same-site request -> 200 accepted



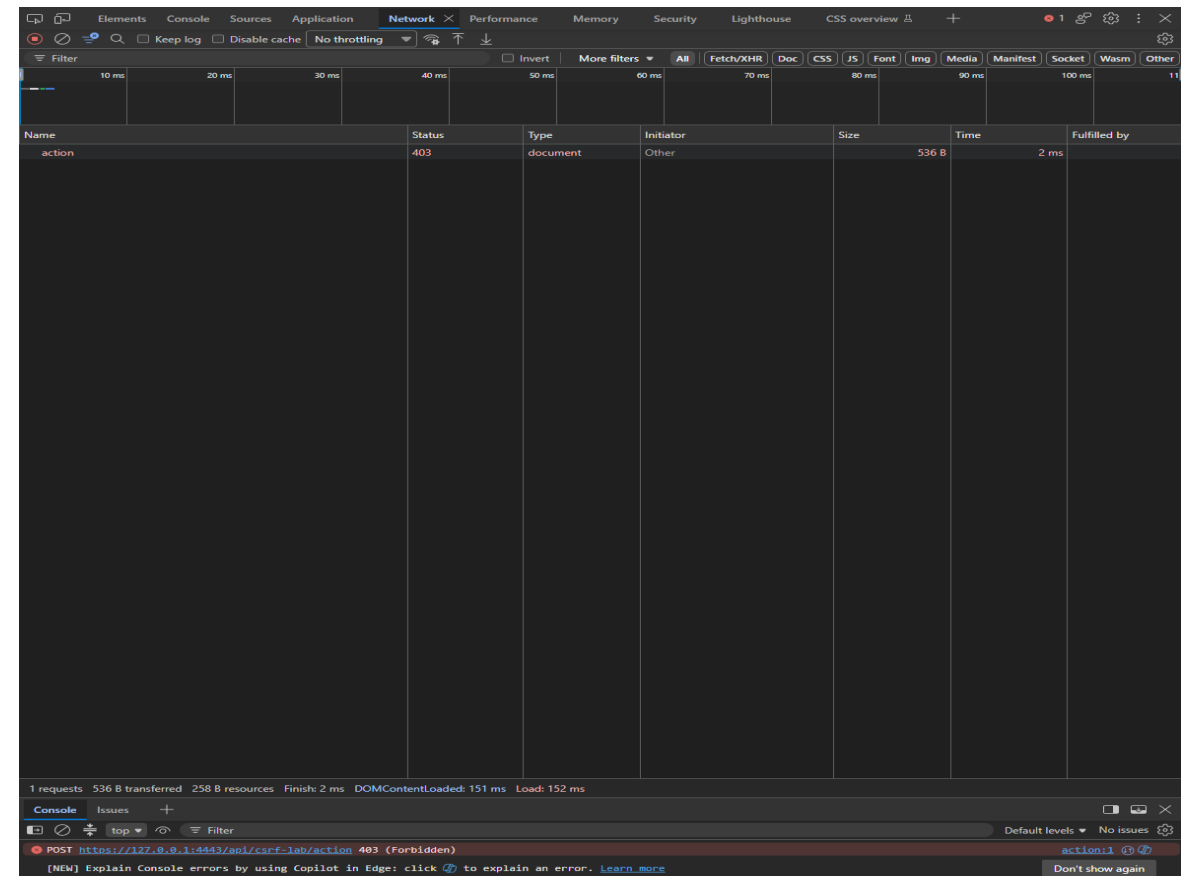
The screenshot shows the Chrome DevTools Network tab with a single request named 'same-site'. The status is 200, the type is document, and the initiator is Other. The size is 555 B and the time is 2 ms. The console shows no issues.

Name	Status	Type	Initiator	Size	Time	Fulfilled by
same-site	200	document	Other	555 B	2 ms	

1 requests 555 B transferred 284 B resources Finish: 2 ms DOMContentLoaded: 173 ms Load: 174 ms

Console: Issues +

Cross-site request -> 403 blocked



The screenshot shows the Chrome DevTools Network tab with a single request named 'action'. The status is 403, the type is document, and the initiator is Other. The size is 536 B and the time is 2 ms. The console shows an error: 'POST https://127.0.0.1:4443/api/csrf-lab/action 403 (Forbidden)'.

Name	Status	Type	Initiator	Size	Time	Fulfilled by
action	403	document	Other	536 B	2 ms	

1 requests 536 B transferred 258 B resources Finish: 2 ms DOMContentLoaded: 151 ms Load: 152 ms

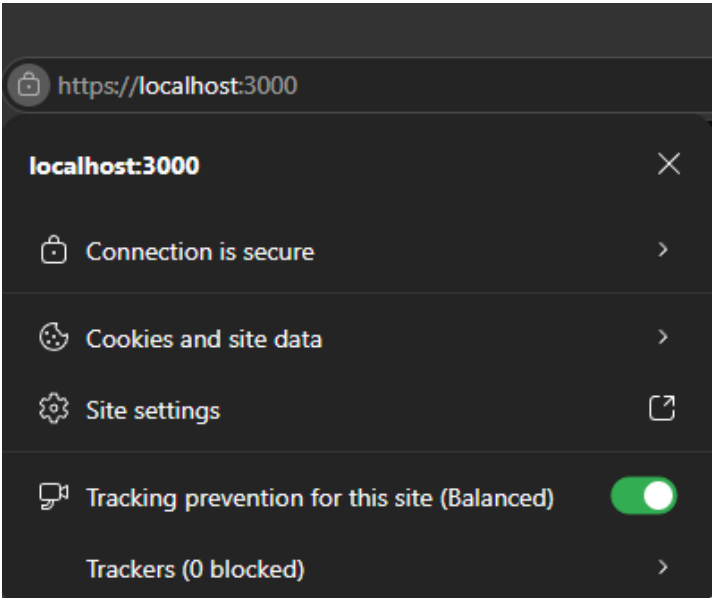
Console: Issues +

POST https://127.0.0.1:4443/api/csrf-lab/action 403 (Forbidden)

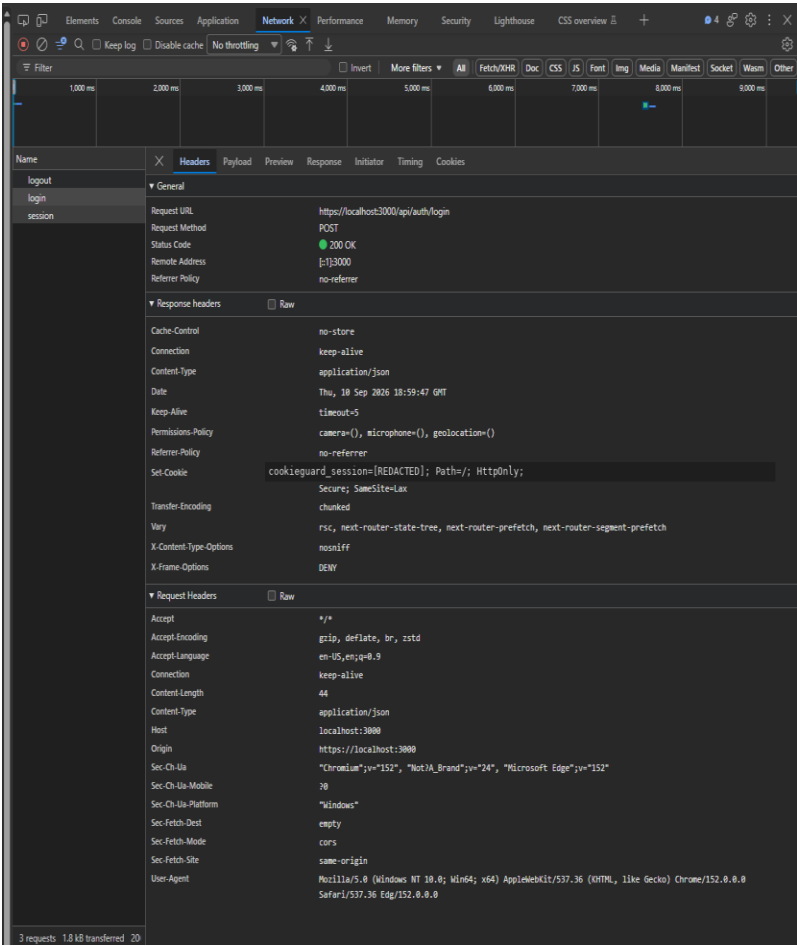
[NEW] Explain Console errors by using Copilot in Edge: click to explain an error. Learn more

Don't show again

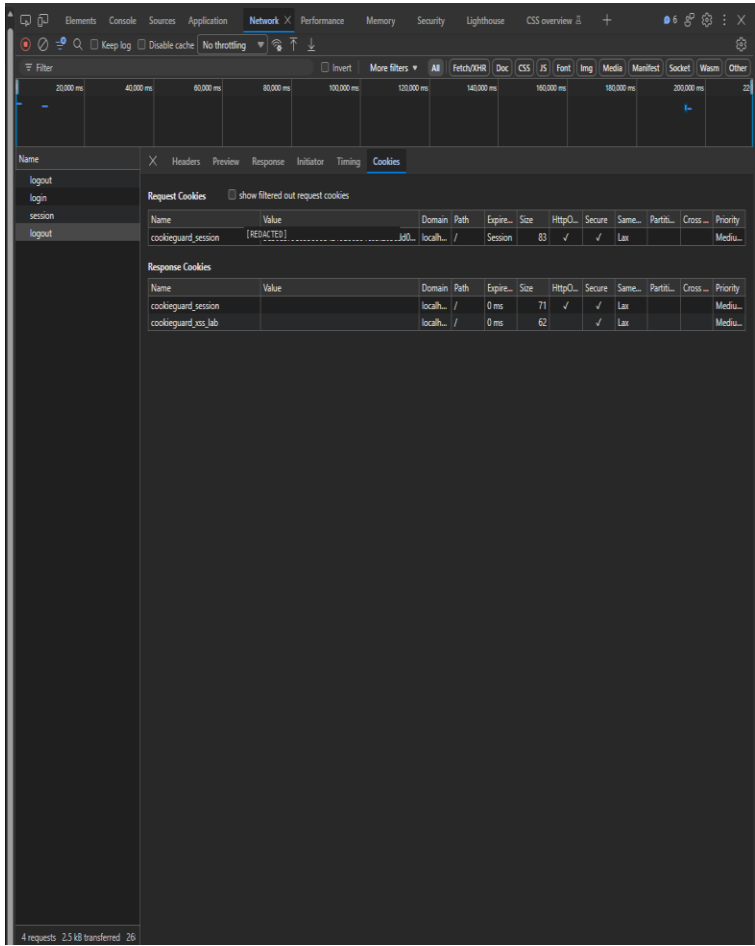
HTTPS and secure transport



HTTPS confirmed on the frontend origin



Login request Network evidence (Set-Cookie)



Logout clears the session cookie

Results and security takeaways

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Owner\apps\CookieGuard> npm run verify
npm notice run cookieguard@0.1.0 verify
npm notice run node scripts/verify.mjs

=== Backend unit and security tests ===
npm notice run @cookieguard/backend@0.1.0 test
npm notice run node --import tsx --test tests/*.test.ts
✓exposes the expected session cookie attributes (0.4441ms)
✓provides an explanation for every inspected attribute (0.1094ms)
✓builds the same secure cookie attributes used by the login response (0.1213ms)
✓builds a session-cookie clearing header with matching security attributes (0.0541ms)
✓documents that Secure requires HTTPS in the current lab (0.0643ms)
✓builds a secure Lax CSRF lab cookie (0.6055ms)
✓builds a secure Strict CSRF lab cookie (0.1563ms)
✓generates a fresh CSRF lab cookie value (0.1437ms)
✓detects the CSRF lab cookie (0.075ms)
✓builds a secure clearing header (0.0661ms)
✓HTTPS development certificate paths use the shared certs directory (0.4669ms)
✓creates and retrieves a server-side session (1.1422ms)
✓deletes a server-side session (0.2051ms)
✓builds the vulnerable XSS lab cookie without HttpOnly (0.4689ms)
✓builds the protected XSS lab cookie with HttpOnly (0.0813ms)
✓generates a fresh XSS lab cookie value (0.2319ms)
✓builds a clearing header for the XSS lab cookie (0.0571ms)
i tests 17
i suites 0
i pass 17
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 233.5188

=== Backend TypeScript build ===
npm notice run @cookieguard/backend@0.1.0 build
npm notice run tsc -p tsconfig.json
```

Verification results

- ✓ 17 / 17 backend tests passed, 0 failures
- ✓ Backend TypeScript build passed
- ✓ Frontend production build passed
- ✓ GitHub Actions CI: Verify CookieGuard #64 - success
- ✓ HTTPS, Secure, HttpOnly, SameSite verified
- ✓ XSS and CSRF labs verified
- ✓ Logout and session clearing verified

Limitations

- Fixed demo credentials, not a real user database
- In-memory sessions only
- Local mkcert HTTPS certificate
- Controlled vulnerable XSS mode is intentional
- Dependency maintenance (Next.js) needed before any production use